

SQL Query Fundamentals

1. Overview

SQL, or Structured Query Language, is the standard language used to store, retrieve, filter, group, combine, and analyze data in relational databases.

Definition

SQL is a declarative language where you describe **what data you want**, and the database decides **how to fetch it efficiently**.

Example:

```
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

This query says: "Give me employee names and salaries where salary is greater than 50000."

Why It Matters

SQL matters because almost every real-world application stores data:

Application	Data Stored
E-commerce	users, products, orders, payments
Banking	accounts, transactions, balances
Social media	users, posts, likes, comments
Placement portals	students, companies, applications
Ride sharing	drivers, riders, trips, payments

Backend engineers frequently need to write correct and efficient SQL queries.

Where It Is Used in Real Systems

SQL is used in:

- Backend APIs
- Data analytics dashboards
- Reporting systems
- Payment systems
- Admin panels
- Recommendation systems
- Data validation and cleanup

- Online assessment questions

Why Interviewers Ask About It

Interviewers ask SQL because it tests:

- Data filtering using **WHERE**
- Aggregation using **GROUP BY**
- Post-aggregation filtering using **HAVING**
- Combining tables using joins
- Nested logic using subqueries
- Ranking and analytics using window functions
- Conditional logic using **CASE WHEN**
- Ability to reason from table structure to output

SQL questions are common in SDE placements because backend developers must understand data retrieval deeply.

2. Core Idea

The core idea of SQL queries is:

Start from tables, filter rows, combine data, group data, calculate results, and return the required output.

Intuition

Think of a database table like an Excel sheet.

id	name	department	salary
1	Asha	IT	70000
2	Ravi	HR	45000
3	Neha	IT	80000

SQL lets you ask questions like:

- Which employees are in IT?
- What is the average salary per department?
- Which department has more than 5 employees?
- Who earns more than the department average?

Real-World Analogy

Imagine a college placement office.

They have multiple registers:

- Students register
- Companies register

- Applications register
- Offers register

SQL helps answer questions like:

- Which students applied to Amazon?
- Which companies offered packages above 10 LPA?
- How many students were placed per department?
- Which students received more than one offer?

Small Example

Suppose we have a table:

orders

order_id	customer_id	amount	status
1	101	500	completed
2	102	800	cancelled
3	101	1200	completed
4	103	700	completed

Question: Find total completed order amount per customer.

```
SELECT customer_id, SUM(amount) AS total_amount
FROM orders
WHERE status = 'completed'
GROUP BY customer_id;
```

Step-by-Step Explanation

Logical query processing order:

Step	Clause	Meaning
1	FROM	Choose table
2	JOIN	Combine tables
3	WHERE	Filter rows
4	GROUP BY	Form groups
5	HAVING	Filter groups
6	SELECT	Choose output columns
7	ORDER BY	Sort final result

Step	Clause	Meaning
8	LIMIT	Restrict number of rows

Important interview trap:

The written order starts with **SELECT**, but the logical execution starts with **FROM**.

3. Important Subtopics

3.1 SELECT

What It Means

SELECT chooses the columns or expressions to display in the final result.

```
SELECT name, salary
FROM employees;
```

Why It Matters

It controls the output of the query. In interviews, you must return only the required columns.

Example

```
SELECT name, salary * 12 AS annual_salary
FROM employees;
```

Common Interview Angle

Interviewers check whether you understand:

- Selecting specific columns
- Using aliases
- Selecting computed values
- Avoiding unnecessary **SELECT ***

Common mistake:

```
SELECT *
FROM employees;
```

Using **SELECT *** is usually bad in production because it fetches unnecessary data and can break code if schema changes.

3.2 WHERE

What It Means

WHERE filters rows before grouping or aggregation.

```
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

Why It Matters

Most queries require filtering. **WHERE** reduces the number of rows processed.

Example

```
SELECT *
FROM orders
WHERE status = 'completed'
AND amount > 1000;
```

Common Interview Angle

Interviewers test:

- **AND, OR, NOT**
- **IN**
- **BETWEEN**
- **LIKE**
- **IS NULL**
- Difference between **WHERE** and **HAVING**

Important:

```
WHERE column = NULL
```

is wrong. Use:

```
WHERE column IS NULL
```

3.3 GROUP BY

What It Means

GROUP BY combines rows with the same value into groups so aggregate functions can be applied.

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

Why It Matters

It is used for reports and analytics:

- Count users by city
- Total sales by month
- Average salary by department
- Number of orders per customer

Example

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id;
```

Common Interview Angle

Interviewers check whether you know:

- Every non-aggregated selected column should usually appear in **GROUP BY**
- Aggregates like **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**
- Grouping happens after **WHERE**

Wrong:

```
SELECT department, name, AVG(salary)
FROM employees
GROUP BY department;
```

Why wrong:

name is neither grouped nor aggregated.

3.4 HAVING

What It Means

HAVING filters groups after **GROUP BY**.

```
SELECT department, COUNT(*) AS employee_count
FROM employees
```

```
GROUP BY department
HAVING COUNT(*) > 5;
```

Why It Matters

WHERE cannot filter aggregate results. **HAVING** can.

Example

Find customers who placed more than 3 orders:

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 3;
```

Common Interview Angle

Most asked comparison:

WHERE	HAVING
Filters rows	Filters groups
Runs before GROUP BY	Runs after GROUP BY
Cannot directly use aggregate conditions	Can use aggregate conditions
Used for raw row filters	Used for grouped result filters

3.5 Joins

What It Means

Joins combine rows from two or more tables using a related column.

Example tables:

```
students
-----
student_id | name
-----
1          | Asha
2          | Ravi

marks
-----
student_id | subject | score
-----
```

1	SQL	90
2	SQL	75

Query:

```
SELECT s.name, m.subject, m.score
FROM students s
JOIN marks m
ON s.student_id = m.student_id;
```

Why It Matters

Real databases are normalized. Data is split across tables to avoid duplication.

Types of Joins

Join Type	Meaning
INNER JOIN	Returns only matching rows
LEFT JOIN	Returns all rows from left table and matching rows from right
RIGHT JOIN	Returns all rows from right table and matching rows from left
FULL OUTER JOIN	Returns all rows from both tables
CROSS JOIN	Returns all combinations
SELF JOIN	Joins a table with itself

Example

Find all students and their marks, including students with no marks:

```
SELECT s.name, m.score
FROM students s
LEFT JOIN marks m
ON s.student_id = m.student_id;
```

Common Interview Angle

Interviewers test:

- Difference between **INNER JOIN** and **LEFT JOIN**
- Join condition placement
- Handling unmatched rows
- Duplicate rows after joining
- Self joins for manager-employee problems

3.6 Subqueries

What It Means

A subquery is a query inside another query.

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Why It Matters

Subqueries help solve problems in steps.

Example

Find employees earning more than the company average:

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Common Interview Angle

Interviewers check:

- Scalar subqueries
- Multi-row subqueries with **IN**
- Correlated subqueries
- Subquery versus join
- Performance implications

Correlated subquery example:

```
SELECT e1.name, e1.department, e1.salary
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e1.department
);
```

This finds employees earning more than their department average.

3.7 Window Functions

What It Means

Window functions perform calculations across related rows without collapsing rows like `GROUP BY`.

```
SELECT name,
       department,
       salary,
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Why It Matters

Window functions are used for:

- Ranking
- Running totals
- Moving averages
- Top N per group
- Comparing current row with previous row

Example

Find top 2 highest-paid employees per department:

```
SELECT *
FROM (
    SELECT name,
           department,
           salary,
           DENSE_RANK() OVER (
               PARTITION BY department
               ORDER BY salary DESC
           ) AS rnk
    FROM employees
)
```

```
) ranked  
WHERE rnk <= 2;
```

Common Interview Angle

Common functions:

Function	Use
<code>ROW_NUMBER()</code>	Unique row number
<code>RANK()</code>	Ranking with gaps
<code>DENSE_RANK()</code>	Ranking without gaps
<code>LAG()</code>	Previous row value
<code>LEAD()</code>	Next row value
<code>SUM() OVER</code>	Running or partitioned total
<code>AVG() OVER</code>	Moving or partitioned average

Interviewers often ask top N per group, second highest salary, running total, and duplicate removal.

3.8 CASE WHEN

What It Means

`CASE WHEN` adds conditional logic inside SQL.

```
SELECT name,  
       salary,  
       CASE  
         WHEN salary >= 80000 THEN 'High'  
         WHEN salary >= 50000 THEN 'Medium'  
         ELSE 'Low'  
       END AS salary_band  
FROM employees;
```

Why It Matters

It is used to categorize data, create flags, and write conditional aggregations.

Example

Count completed and cancelled orders:

```
SELECT  
  SUM(CASE WHEN status = 'completed' THEN 1 ELSE 0 END) AS completed_orders,
```

```
SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) AS cancelled_orders
FROM orders;
```

Common Interview Angle

Interviewers test:

- Conditional columns
- Conditional aggregation
- Replacing multiple queries with one query
- Data bucketing

4. Real-World Example

Placement Portal Database

Suppose a placement portal has these tables:

```
students
-----
student_id | name | department | graduation_year

companies
-----
company_id | name | package_lpa

applications
-----
application_id | student_id | company_id | status
```

Question:

Find departments where more than 10 students received offers, and show the average package.

```
SELECT s.department,
       COUNT(DISTINCT s.student_id) AS placed_students,
       AVG(c.package_lpa) AS avg_package
FROM students s
JOIN applications a
  ON s.student_id = a.student_id
JOIN companies c
  ON a.company_id = c.company_id
WHERE a.status = 'offered'
GROUP BY s.department
HAVING COUNT(DISTINCT s.student_id) > 10
ORDER BY avg_package DESC;
```

What This Uses

Concept	Usage
SELECT	Choose final columns
JOIN	Combine students, applications, companies
WHERE	Keep only offered applications
GROUP BY	Group by department
HAVING	Keep departments with more than 10 placed students
ORDER BY	Sort by package

5. Diagrams / Mental Models

SQL Logical Flow

```
FROM
|
v
JOIN
|
v
WHERE
|
v
GROUP BY
|
v
HAVING
|
v
SELECT
|
v
ORDER BY
|
v
LIMIT
```

Join Mental Model

```
INNER JOIN
Left table matched with right table only

LEFT JOIN
All left rows + matching right rows
Unmatched right columns become NULL

FULL OUTER JOIN
```

Everything from both sides
Unmatched parts become NULL

GROUP BY Mental Model

Rows:
IT, 70000
HR, 40000
IT, 80000
HR, 50000

GROUP BY department:
IT -> 70000, 80000
HR -> 40000, 50000

Aggregate:
IT -> AVG = 75000
HR -> AVG = 45000

Window Function Mental Model

GROUP BY:
Many rows become one row per group.

Window function:
Rows stay as they are, but each row gets extra calculated information.

6. Common Interview Questions

1. What is SQL?

SQL is a language used to query and manage relational databases. It allows us to retrieve, filter, insert, update, delete, group, and join data.

Key points interviewer expects:

- SQL is declarative
- Used with relational databases
- Works on tables, rows, and columns
- Used for data retrieval and manipulation

Common mistakes:

- Saying SQL is only for fetching data
- Confusing SQL with a specific database like MySQL

2. What is the difference between **WHERE** and **HAVING**?

WHERE filters individual rows before grouping. **HAVING** filters grouped results after aggregation.

Example:

```
SELECT department, COUNT(*)
FROM employees
WHERE salary > 50000
GROUP BY department
HAVING COUNT(*) > 5;
```

Key points interviewer expects:

- **WHERE** before **GROUP BY**
- **HAVING** after **GROUP BY**
- Aggregate conditions usually go in **HAVING**

Common mistakes:

- Using aggregate functions directly in **WHERE**

3. What is the difference between **INNER JOIN** and **LEFT JOIN**?

INNER JOIN returns only matching rows from both tables. **LEFT JOIN** returns all rows from the left table and matching rows from the right table. If no match exists, right table columns become **NULL**.

Key points interviewer expects:

- Matching versus preserving all left rows
- **NULL** for unmatched rows in **LEFT JOIN**
- Use **LEFT JOIN** when missing related data should still be shown

Common mistakes:

- Thinking **LEFT JOIN** returns only unmatched rows

4. What is **GROUP BY** used for?

GROUP BY groups rows with the same values so aggregate functions can be applied.

Example:

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

Key points interviewer expects:

- Groups rows
- Used with aggregates

- Non-aggregated selected columns should be grouped

Common mistakes:

- Selecting columns that are neither grouped nor aggregated

5. What is a subquery?

A subquery is a query inside another query. It can be used in **SELECT**, **FROM**, or **WHERE**.

Example:

```
SELECT name
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Key points interviewer expects:

- Inner query result is used by outer query
- Can return one value, multiple values, or a table
- Can be correlated or non-correlated

Common mistakes:

- Assuming all subqueries are slow
- Using **=** with a subquery that returns multiple rows

6. What is a correlated subquery?

A correlated subquery depends on the current row of the outer query.

Example:

```
SELECT e1.name
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e1.department
);
```

Key points interviewer expects:

- Inner query references outer query
- Often evaluated per outer row conceptually
- Useful for row-specific comparisons

Common mistakes:

- Not noticing the outer alias inside the subquery

7. What is the difference between **RANK**, **DENSE_RANK**, and **ROW_NUMBER**?

Function	Behavior
ROW_NUMBER()	Gives unique sequence numbers
RANK()	Same rank for ties, leaves gaps
DENSE_RANK()	Same rank for ties, no gaps

Example scores:

name	score	ROW_NUMBER	RANK	DENSE_RANK
A	100	1	1	1
B	90	2	2	2
C	90	3	2	2
D	80	4	4	3

Key points interviewer expects:

- Ties matter
- **RANK** leaves gaps
- **DENSE_RANK** does not leave gaps

Common mistakes:

- Using **ROW_NUMBER** when ties should be preserved

8. How do you find the second highest salary?

Using **DENSE_RANK**:

```
SELECT salary
FROM (
    SELECT salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) x
WHERE rnk = 2;
```

Key points interviewer expects:

- Handles duplicate highest salaries
- **DENSE_RANK** is usually better than **LIMIT OFFSET** for this question

Common mistakes:

- Using `LIMIT 1 OFFSET 1` without considering duplicates

9. What is `CASE WHEN` used for?

`CASE WHEN` is used for conditional logic inside SQL.

Example:

```
SELECT name,  
       CASE  
         WHEN marks >= 90 THEN 'A'  
         WHEN marks >= 75 THEN 'B'  
         ELSE 'C'  
       END AS grade  
FROM students;
```

Key points interviewer expects:

- Creates conditional output
- Can be used inside aggregate functions
- Similar to if-else logic

Common mistakes:

- Forgetting `END`
- Not handling the `ELSE` case

10. What is the difference between `COUNT(*)` and `COUNT(column)`?

`COUNT(*)` counts all rows. `COUNT(column)` counts only rows where that column is not `NULL`.

Example:

```
SELECT COUNT(*), COUNT(email)  
FROM users;
```

Key points interviewer expects:

- `COUNT(column)` ignores `NULL`
- `COUNT(*)` does not ignore rows

Common mistakes:

- Thinking both always return the same result

11. What is the difference between `IN` and `EXISTS`?

IN checks whether a value exists in a result set. **EXISTS** checks whether a subquery returns at least one row.

Example:

```
SELECT name
FROM students s
WHERE EXISTS (
    SELECT 1
    FROM applications a
    WHERE a.student_id = s.student_id
);
```

Key points interviewer expects:

- **EXISTS** is useful for correlated checks
- **IN** is simple for matching values
- **NULL** behavior can differ

Common mistakes:

- Ignoring **NULL** behavior with **NOT IN**

12. How do you get top N records per group?

Use a window function:

```
SELECT *
FROM (
    SELECT employee_id,
           department,
           salary,
           ROW_NUMBER() OVER (
               PARTITION BY department
               ORDER BY salary DESC
           ) AS rn
    FROM employees
) x
WHERE rn <= 3;
```

Key points interviewer expects:

- Use **PARTITION BY**
- Use ranking or row numbering
- Filter outside the window function query

Common mistakes:

- Using global **LIMIT**, which gives top N overall, not per group

7. Deep-Dive Questions

1. Why can window functions solve problems that **GROUP BY** cannot solve cleanly?

GROUP BY collapses rows. Window functions keep the original rows and add calculations over related rows.

Example:

```
SELECT name,
       department,
       salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM employees;
```

This shows each employee along with their department average. With **GROUP BY**, individual employee rows would be lost.

2. What happens if a **LEFT JOIN** condition is placed in **WHERE** instead of **ON**?

It can accidentally turn the **LEFT JOIN** into an **INNER JOIN**.

Problem:

```
SELECT s.name, a.status
FROM students s
LEFT JOIN applications a
  ON s.student_id = a.student_id
WHERE a.status = 'offered';
```

Students with no applications are removed because **a.status** is **NULL**.

Better if you want all students and only offered application matches:

```
SELECT s.name, a.status
FROM students s
LEFT JOIN applications a
  ON s.student_id = a.student_id
AND a.status = 'offered';
```

3. How do you find duplicate records?

Use **GROUP BY** and **HAVING**.

```
SELECT email, COUNT(*) AS count_email
FROM users
```

```
GROUP BY email
HAVING COUNT(*) > 1;
```

If you need full duplicate rows, use a window function:

```
SELECT *
FROM (
    SELECT u.*,
           COUNT(*) OVER (PARTITION BY email) AS email_count
    FROM users u
) x
WHERE email_count > 1;
```

4. How can CASE WHEN be used for conditional aggregation?

It can count or sum only rows matching a condition.

```
SELECT
    department,
    SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END) AS female_count,
    SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END) AS male_count
FROM employees
GROUP BY department;
```

This avoids writing separate queries for each condition.

5. How do indexes affect SQL query performance?

Indexes help the database find rows faster, especially for columns used in:

- WHERE
- JOIN
- ORDER BY
- GROUP BY

But indexes also have costs:

- Extra storage
- Slower inserts and updates
- Maintenance overhead

Interview answer:

Indexes are like a book index. They speed up lookups but need extra space and must be updated when data changes.

8. Comparison Tables

WHERE vs HAVING

Feature	WHERE	HAVING
Filters	Rows	Groups
Used before aggregation	Yes	No
Used after aggregation	No	Yes
Can use aggregate functions directly	Usually no	Yes
Example	<code>WHERE salary > 50000</code>	<code>HAVING COUNT(*) > 5</code>

GROUP BY vs Window Functions

Feature	GROUP BY	Window Functions
Row count after operation	Reduces rows	Preserves rows
Main use	Summary per group	Analytics per row
Can show individual row details	Not directly	Yes
Example	Average salary per department	Employee salary with department average

INNER JOIN vs LEFT JOIN

Feature	INNER JOIN	LEFT JOIN
Returns matched rows	Yes	Yes
Returns unmatched left rows	No	Yes
Missing right-side values	Not included	<code>NULL</code>
Use case	Only related data needed	Keep all left records

Subquery vs Join

Feature	Subquery	Join
Style	Query inside query	Combines tables directly
Readability	Good for step-by-step logic	Good for relational data
Performance	Depends on optimizer	Often efficient for combining data
Common use	Filtering by derived result	Fetching columns from multiple tables

RANK vs DENSE_RANK vs ROW_NUMBER

Function	Ties Get Same Number	Gaps After Ties	Unique Number Per Row
<code>ROW_NUMBER()</code>	No	No	Yes
<code>RANK()</code>	Yes	Yes	No
<code>DENSE_RANK()</code>	Yes	No	No

CASE WHEN vs WHERE

Feature	CASE WHEN	WHERE
Purpose	Creates conditional values	Filters rows
Removes rows	No	Yes
Used in <code>SELECT</code>	Yes	No
Used for bucketing	Yes	No

9. Common Mistakes

1. Using `WHERE` for aggregate conditions.

Wrong:

```
WHERE COUNT(*) > 5
```

Correct:

```
HAVING COUNT(*) > 5
```

2. Forgetting that `NULL` is not equal to anything.

Wrong:

```
WHERE manager_id = NULL
```

Correct:

```
WHERE manager_id IS NULL
```

3. Using `SELECT *` in production queries.

It fetches unnecessary data and can make APIs slower.

4. Forgetting join conditions.

Wrong:

```
SELECT *  
FROM students, marks;
```

This may create a Cartesian product.

5. Using `LIMIT` for second highest salary without handling duplicates.
6. Confusing `COUNT(*)` and `COUNT(column)`.
7. Filtering right-table columns in `WHERE` after a `LEFT JOIN`.
8. Selecting non-grouped columns with aggregates.
9. Thinking window functions and `GROUP BY` are the same.
10. Not using aliases in multi-table queries.
11. Using `NOT IN` without considering `NULL`.
12. Forgetting that SQL logical execution order differs from written order.

10. Edge Cases / Special Cases

NULL Handling

`NULL` means unknown or missing value.

```
WHERE column IS NULL  
WHERE column IS NOT NULL
```

Do not use:

```
WHERE column = NULL
```

NOT IN with NULL

This can produce unexpected results:

```
SELECT *  
FROM employees  
WHERE department_id NOT IN (  
    SELECT department_id
```

```
FROM departments  
);
```

If the subquery returns **NULL**, the result may be empty. Prefer **NOT EXISTS** in many cases.

Duplicate Rows After Join

If one student has multiple applications, joining students with applications creates multiple rows for that student.

Use **DISTINCT**, aggregation, or window functions depending on the requirement.

COUNT Behavior

Expression	Meaning
COUNT (*)	Counts all rows
COUNT(column)	Counts non-NULL values
COUNT(DISTINCT column)	Counts unique non-NULL values

Aggregate Functions Ignore NULL

SUM, **AVG**, **MIN**, and **MAX** usually ignore **NULL**.

Filtering Window Function Results

You usually cannot use a window function directly in **WHERE**.

Wrong:

```
SELECT name,  
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn  
FROM employees  
WHERE rn <= 3;
```

Correct:

```
SELECT *  
FROM (  
    SELECT name,  
           ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn  
    FROM employees  
) x  
WHERE rn <= 3;
```

HAVING Without GROUP BY

Some databases allow **HAVING** without explicit **GROUP BY**, treating the entire result as one group.

Cross Join Explosion

If table A has 1000 rows and table B has 1000 rows, **CROSS JOIN** creates 1,000,000 rows.

11. How to Explain in Interview

SQL is a declarative language used to query relational databases. In a typical query, we choose data from tables using **FROM**, combine related tables using joins, filter raw rows using **WHERE**, group rows using **GROUP BY**, filter grouped results using **HAVING**, and finally select the required output columns using **SELECT**. For analytics problems like ranking or top N per group, window functions are useful because they calculate values across related rows without collapsing the original rows. **CASE WHEN** is used when we need conditional logic inside the query.

12. Quick Revision Notes

Key Definitions

Term	Meaning
SELECT	Chooses output columns
WHERE	Filters rows before grouping
GROUP BY	Groups rows for aggregation
HAVING	Filters groups after aggregation
JOIN	Combines tables
Subquery	Query inside another query
Window function	Calculates across rows without collapsing them
CASE WHEN	Conditional logic in SQL

Important Points

- SQL written order is not the same as logical execution order.
- **WHERE** filters rows; **HAVING** filters groups.
- **GROUP BY** collapses rows.
- Window functions preserve rows.
- **LEFT JOIN** keeps all rows from the left table.
- **COUNT(column)** ignores **NULL**.
- **COUNT(*)** counts all rows.
- Use **IS NULL**, not **= NULL**.
- Use **DENSE_RANK** for second highest salary with duplicates.
- Use **PARTITION BY** for ranking within each group.

Common Comparisons

Comparison	Core Difference
<code>WHERE</code> vs <code>HAVING</code>	Row filter vs group filter
<code>INNER JOIN</code> vs <code>LEFT JOIN</code>	Only matches vs all left rows
<code>GROUP BY</code> vs window function	Collapses rows vs keeps rows
<code>RANK</code> vs <code>DENSE_RANK</code>	Gaps vs no gaps
Subquery vs join	Nested logic vs table combination

Must-Remember Facts

- `FROM` logically runs before `SELECT`.
- Aggregates usually cannot be used in `WHERE`.
- `NULL` breaks normal equality comparisons.
- Missing join conditions can create huge Cartesian products.
- Window functions are common in placement SQL rounds.

Interview Traps

- Second highest salary with duplicates
- Top N per group
- `LEFT JOIN` filtered incorrectly in `WHERE`
- `NOT IN` with `NULL`
- `COUNT(*)` vs `COUNT(column)`
- `RANK` vs `DENSE_RANK`

13. Practice Tasks

Use these sample tables:

```
employees(employee_id, name, department_id, salary, manager_id)
departments(department_id, department_name)
orders(order_id, customer_id, order_date, amount, status)
customers(customer_id, name, city)
students(student_id, name, department, cgpa)
applications(application_id, student_id, company_id, status)
companies(company_id, name, package_lpa)
```

Basic Query Practice

1. Write a query to fetch all employees with salary greater than 50000.
2. Write a query to fetch employees from department 10 sorted by salary descending.
3. Write a query to find customers from either Delhi or Mumbai.
4. Write a query to find orders placed between two dates.

GROUP BY and HAVING Practice

5. Find average salary per department.
6. Find departments having more than 5 employees.
7. Find customers whose total order amount is greater than 10000.
8. Find the number of completed and cancelled orders.

Join Practice

9. Fetch employee names with their department names.
10. Fetch all departments and employee names, including departments with no employees.
11. Find students who applied to at least one company.
12. Find students who never applied to any company.

Subquery Practice

13. Find employees earning more than the company average salary.
14. Find employees earning more than their department average salary.
15. Find customers who placed at least one completed order.
16. Find companies with package greater than the average package.

Window Function Practice

17. Find the second highest salary.
18. Find the top 3 salaries in each department.
19. Assign rank to students by CGPA within each department.
20. Calculate running total of order amount by order date.
21. Find each employee's salary and department average salary.

CASE WHEN Practice

22. Categorize employees as **High**, **Medium**, or **Low** salary.
23. Count orders by status using conditional aggregation.
24. Mark students as **Eligible** if CGPA is at least 7.0, otherwise **Not Eligible**.
25. Count how many companies offer package above 10 LPA and below or equal to 10 LPA.

14. Final Cheat Sheet

Core Definition

SQL is a declarative language used to query and manage relational data stored in tables.

Why It Matters

SQL is essential for backend development, analytics, reporting, dashboards, and real-world data-driven applications.

Most Asked Questions

Question	Best Concept
Second highest salary	DENSE_RANK , subquery

Question	Best Concept
Top N per group	Window function with PARTITION BY
Department-wise average salary	GROUP BY
Departments with more than 5 employees	GROUP BY + HAVING
Employees with department names	JOIN
Employees above average salary	Subquery
Count conditionally	CASE WHEN inside SUM
Find duplicates	GROUP BY + HAVING

Common Comparisons

Topic	One-Line Difference
WHERE vs HAVING	WHERE filters rows, HAVING filters groups
INNER JOIN vs LEFT JOIN	Inner keeps matches, left keeps all left rows
GROUP BY vs window function	Grouping reduces rows, window functions preserve rows
RANK vs DENSE_RANK	RANK skips numbers after ties, DENSE_RANK does not
COUNT(*) vs COUNT(column)	All rows vs non-NULL values

One-Line Interview Answer

SQL helps us query relational data by selecting columns, filtering rows, joining tables, grouping and aggregating results, filtering groups, using subqueries for nested logic, window functions for analytics, and **CASE WHEN** for conditional output.